THE UNIVERSITY OF HONG KONG

School of Computing and Data Science

MSc(CompSc) — COMP7705 Capstone Project

OptiTrade Copilot

An AI-Driven Trading Portal with Interactive Dynamic Canvas

INTERIM REPORT

Group: MSc25-COMP7705

Academic Year: 2025/2026

Group Members

Wong Wai Wing (3036383513)

Ng Wing Yin (3036383927)

Hui Nga Chi (3036198774)

Ng Sui Yat (3036383549)

Cheung Ching Nam (3036382959)

Project Mentor: Dr. Chuan Wu

Submission Date: June 1, 2026

Abstract

OptiTrade Copilot is an AI-driven trading portal that addresses the three problems identified in the Detailed Project Proposal: non-personalised dashboards causing information overload (P1), LLM hallucinations in conversational interfaces (P2), and limited personalised monitoring and notification capability (P3). The system combines (i) a Dynamic Customisable Widget Canvas hosting chart, portfolio, and news widgets, (ii) a Context-Aware AI Chat Panel grounded in user-exported widget snapshots, and (iii) a real-time, position-aware alert and notification engine.

At the time of submission, the team has completed the proposal-defined Phase 0 (proposal and setup) and Phase 1 (core infrastructure and MVP widgets), and is toward the completion of Phase 2 (AI features and the News widget). Phase 3, covering the Alerts module, evaluation, and final integration, is scheduled in §5 of this report; detailed line-item status by module is given in §4.3.

Module status is summarised below; each module lead owns the corresponding detail in §4.2.

- **Dynamic Widget Canvas** has completed its Phase 1 MVP — a device-aware responsive grid, drag, drop and resize, multi-layout presets, and versioned layout persistence — and now hosts every data widget through a shared layout contract and chat-context export interface.
- **Chart Widget** has completed Phase 1 (candlestick rendering, three-ticker watchlist, timeframe and interval controls, and Moving Average / Bollinger Bands / RSI overlays) and the Phase 2 Momentum Snapshot pipeline that feeds deterministic return metrics and a technical snapshot to the stock-chart AI endpoint for structured Overview / Momentum / Indicators / Levels commentary; MACD, volume bars, and the explicit Overbought / Oversold / Neutral badge are retained as Phase 3 items.
- **Portfolio Widget** has completed its Phase 1 MVP — small, medium, and large layouts, holdings and P/L summaries, a backend-backed broker-mock path, a broker-connection panel, and persistent paper-portfolio editing — and Phase 2 has added the first portfolio insight layer through a deterministic risk flag, insight card, top-holdings visualisation, and chat-context export. Phase 3 will focus on real-broker connectivity and LLM-grounded portfolio reasoning.
- **News Widget** has completed Phase 1 baseline RSS aggregation and is mid-way through Phase 2, with an asynchronous cloud-LLM analysis pipeline backed by a local rule-matching fallback that delivers per-article sentiment scoring, executive summaries, and risk tagging resilient to API rate limits. A pgvector-based retrieval layer is planned for Phase 3.

- **Context-Aware AI Chat Panel (Chatbot)** has reached a working end-to-end prototype that meets four of the five §3.3.4 MVP acceptance criteria — streaming responses, multi-widget context, per-widget Add-to-Context, and SmartRenderer routing between OpenUI Lang and Markdown — with the fifth, explicit inference-labelling on grounded answers, currently implemented as a best-effort prompt instruction and scheduled for a system-prompt-level upgrade and harness-based evaluation in Phase 3.

- **Real-Time Monitoring and Alerts** remains a Phase 3 module; its design (cron-driven monitoring on the Nanobot backend with email and in-dashboard delivery channels) is fixed and its work items are listed in the §5 schedule.

Remaining risks are time-boxed against the Phase 3 schedule in §5, and confidence in delivering the full proposal scope on or before the final report deadline is high.

Table of Contents

1. Introduction

1.1 Background and Motivation

Despite the proliferation of analytics platforms and increasingly capable LLMs, traders continue to operate in a fragmented, noisy, and difficult-to-verify information environment. Portfolio positions, real-time market data, technical charts, and news feeds commonly reside in separate applications; integrating those sources to form a coherent, auditable basis for decisions remains a largely manual task [2][3]. Conversational LLM agents can provide fluent explanations and summaries, but their outputs frequently lack a verifiable link to the specific data a trader sees [6][7]. Generic alerting systems are pre-configured and not tied to a user's holdings or strategy, leading to missed events or alert fatigue [11]. Together these limitations create three concrete problems (P1–P3) that the OptiTrade Copilot system is designed to address.

1.2 Problem Statement

The system targets three problems:

- P1 — Non-personalized dashboards cause information overload. Generic views designed for multiple user types display irrelevant signals, slowing event recognition and increasing error rates on time-sensitive decisions.
- P2 — LLM hallucinations in conversational interfaces. Models lack grounded access to the trader's exact data snapshot, producing unverifiable claims with compliance and auditability exposure.
- P3 — Limited personalized monitoring and notification capability. Coarse, pre-configured alerts not bound to user holdings or strategy lead to missed opportunities or alert fatigue.

Each problem is owned by a distinct module: P1 by the Dynamic Customizable Widget Canvas, P2 by the Context-Aware AI Chat Panel (the Chatbot module covered in detail in this report), and P3 by the Real-Time Monitoring & Alerts module. Assumptions and boundaries: equities/ETFs as the asset class; English-language conversation; a single authenticated user session at a time. Real-money order placement is explicitly out of scope — the system is decision-support, not an execution agent.

1.3 Objectives and Scope

The system pursues three measurable objectives:

Objective	Description	Measurement method
Reduce decision latency	Personalized, strategy-relevant views surface prioritized events faster.	Task completion times in controlled user studies.
Higher alert signal-to-noise	Position-aware, user-customizable monitors reduce irrelevant notifications.	Alert precision/recall against an event corpus or user feedback.
Reduce hallucination rate	Constrain model outputs to cited snapshot contents and provenance markers.	Hallucination rate on benchmarked question sets.

Module-level scope statements are listed in §4; this report's milestone scope is: Phases 0 and 1 complete, Phase 2 in progress (Chat Panel and Widget module to MVP, testing and fine-tuning in progress).

1.4 Summarized Implementation to Date

Implementation are listed by module. Detailed implementation status is in §4.2.

- Dynamic Widget Canvas — Implemented a customizable widget canvas that allows users to edit with drag-and-drop controls, device-like grid system, and local and account-wise persistence; base widget skeleton and logic to integrate with other elements of the web application.
- Chart Widget — Delivered an interactive candlestick widget with a three-ticker watchlist, timeframe/interval controls, responsive sizing, and chart responses. Implemented Moving Average, Bollinger Bands, and RSI, including corresponding backend technical snapshots for AI use. Added the Phase 2 Momentum Snapshot pipeline: deterministic 1-/5-/20-bar return metrics and technical context are passed to the stock-chart AI endpoint to produce structured Overview / Momentum / Indicators / Levels commentary, while deterministic pivot-cluster support/resistance levels, rule-based fallback insight. MACD, volume bars, and tri-state overbought/oversold labels are retained as Phase 3 completion items.
- Portfolio Widget — Delivered Portfolio Full plus compact P/L-oriented layouts, backend-backed snapshot retrieval with live/demo fallback, broker connection UI, and persistent paper-portfolio editing. Added the first portfolio insight layer (risk flag, insight card, and top-holdings visualisation), together with chat-context export and API contract tests for snapshot retrieval, paper-portfolio save, and broker-connection validation.
- News Widget — Implemented a multi-source financial news aggregator with dynamic category filtering; integrated an asynchronous background pipeline

featuring a hybrid cloud-LLM and deterministic rule-matching fallback mechanism to attach real-time sentiment scores, summary badges, and financial risk tags.

- Context-Aware AI Chat Panel (Chatbot) — Set up nanobot as the AI Agentic backend; Integrate with nanobot via WebSocket; Created Context Store that decoupling widgets from chat state; per-widget Add-to-Chat-Context wiring; multi-context cards bar; SmartRenderer routing with OpenUI Lang.

2. Literature Review

This section condenses proposal §2 (Existing Financial Dashboard Platforms; AI Technologies for Financial Analysis; Gap Analysis).

2.1 Existing Financial Dashboard Platforms

Three reference platforms frame the design space (proposal §2.1):

Platform	Key strengths	Limitations addressed by this project
TradingView	Advanced multi-chart layouts, hundreds of indicators, real-time screeners, broker integrations.	No AI-grounded conversational layer; alerts are symbol-centric, not portfolio-aware; no context-card provenance.
Interactive Brokers TWS	Professional execution, customisable Mosaic interface, 100+ order types, strong APIs.	Interface complexity creates high cognitive load; minimal natural-language support; no AI chat or sentiment analysis.
Perplexity Finance	Conversational AI-first, 40+ live finance integrations, brokerage connectivity, AI risk assessments.	Context grounding is opaque (no exportable snapshot cards); limited widget personalization; no rule-based alert authoring.

2.2 AI Technologies for Financial Analysis

Several AI/ML categories are relevant to OptiTrade Copilot:

- Deep learning for market prediction (LSTMs, Transformers, RL) — underpins the Chart Widget's Momentum Snapshot.
- NLP and sentiment analysis (FinBERT, LLM-based sentiment scoring) [12][13] — powers per-article sentiment scores and risk tagging in the News Widget.
- LLM grounding and hallucination mitigation (RAG, context cards, provenance markers) [7][8][9][10] — the core mechanism for the Context-Aware AI Chat Panel (the chatbot module).
- Rule-based and event-driven alerting (threshold monitoring, Celery/Redis task queues, multi-channel notification) [11] — scheduler for the Alerts module.

- Explainable AI — evidence-linked outputs, confidence labels, reasoning transparency [5] — disclaimer and reasoning indicators in the chat panel.

2.3 Chatbot-specific Depth Pass — Grounding and Generative UI

Within the LLM-grounding strand, two patterns dominate:

- Retrieval-Augmented Generation (RAG) [8][9][10] — the system retrieves relevant documents at query time and injects them into the prompt. Effective for unbounded text corpora, but requires a vector store and an embedding pipeline.
- Context-card / explicit-evidence injection — the system injects user-pinned, timestamped snapshots of UI state directly into the prompt under a labelled header (proposal §3.3.2). Lower infrastructure cost; the user explicitly controls what is grounded against.

On the rendering side, the OpenUI project introduces OpenUI Lang, a token-efficient declarative format that lets an LLM emit a tree of UI components which a strongly-typed renderer materialises as React elements. This complements Markdown for cases where a structured layout (e.g., comparison cards, decision matrices) is clearer than prose — directly supporting proposal §3.3.1 and going beyond it.

OptiTrade Copilot adopts the context-card pattern as the primary grounding mechanism for the interim milestone and plans to layer RAG on top of the News widget in Phase 3.

2.4 Research Gap and Project Positioning

Per proposal §2.3, no existing platform combines (a) a fully personalized widget canvas, (b) AI responses explicitly grounded in verifiable on-screen data snapshots, and (c) position-aware, rule-based alert authoring in a single integrated system. OptiTrade Copilot is designed to close all three gaps simultaneously, drawing on research in information overload reduction [1][2], RAG-based hallucination mitigation [8][9][10], and intelligent alert systems [11].

3. Methodology

3.1 Overall Approach

The system follows the three-layer architecture: a Frontend analysis dashboard (widget canvas, chat panel, alert UI), a Backend API & data service (market data, portfolio, news, alert rules), and an AI Platform (LLM inference, context-card store, RAG grounding). A context-management sub-system spans all three layers: widgets export data snapshots, each is assigned a unique contextId and stored, and relevant blocks are injected into LLM prompts at query time. This is the primary mechanism through which the system achieves provenance and hallucination reduction [7][8].

Implementation proceeds in four phases. Phases 0–1 (proposal/setup, core infrastructure & MVP widgets) are complete; Phase 2 (AI features and News widget) is in progress; Phase 3 (alerts, testing, final submission) is scheduled in Section 6 of this report.

3.2 Data Collection / Datasets

The system consumes (a) live market data via broker / market-data provider APIs, (b) news feeds aggregated by the News widget, (c) user portfolio data (mock broker for the interim phase, real broker integration in Phase 3, and (d) small evaluation sets for the chatbot's grounding/hallucination benchmarks.

- Market data — The chart widget is powered by Financial Modeling Prep (FMP), accessed through a thin FastAPI proxy that normalises the response into the OHLC schema consumed by the candlestick renderer and the deterministic technical snapshots
- Portfolio data — For the interim milestone, the widget uses a broker-mock interface persisted as local JSON through the FastAPI portfolio service. The stored schema covers positions (symbol, quantity, average cost, current price), summary metrics, editable paper-portfolio state, broker-connection settings, and intraday history; real broker synchronisation is deferred to Phase 3.
- News feed — The module aggregates real-time streaming data from major financial media webhooks and standardized RSS feeds (including Yahoo Finance and Economic Times). Ingested articles undergo a server-side deduplication filter based on titles and lexical similarity, maintaining a sliding-window cache with a 15-minute refresh cadence. A financial-relevance filter further screens the content, retaining only economy-related and market-moving news before the deduplicated results are fed to the user dashboard.

3.2.1 Chatbot evaluation sets

The chatbot does not train a model and therefore does not require a labelled training corpus. Three small evaluation sets are being curated for Phase 3:

- Grounded-prompt set: ~50 hand-written user prompts paired with one or more synthetic widget snapshots and a reference answer. Used to score grounding faithfulness.
- Hallucination-bait set: ~30 prompts that ask about facts intentionally absent from the pinned context, used to measure how often the model fabricates.
- UI-rendering set: ~20 prompts whose ideal answer is a structured component (e.g., comparison card), used to measure OpenUI emission quality.

All three sets are produced internally and contain no real user data; they will be released alongside the final report subject to mentor approval.

3.3 Tools and Technologies

Component	Technology (interim)	Justification / proposal mapping
Frontend canvas & widgets	Next.js 16, React 19, Tailwind CSS, shadcn/ui, lightweight-charts, Recharts	Matches proposal §3.1 (React, Tailwind, Shadcn UI, charting library).
Frontend chat UI (Chatbot)	react-markdown + remark-gfm; @openuidev/react-lang, @openuidev/react-ui	Markdown rendering satisfies §3.3.1; OpenUI extends it with generative UI for structured answers.
Backend API & data service	Firebase, Next.js API, hybrid REST and gRPC	Maps to proposal §3.1 backend layer; market data, portfolio, news, alert rule storage.
Real-time transport	Native WebSocket	By committing to WebSocket; used by both the chatbot stream and the in-dashboard alert push.
LLM hosting (Chatbot)	Nanobot agent server (interim) at ws://178.128.213.162:8765	Provider-agnostic streaming endpoint for the interim phase; replaces the “Azure OpenAI + LangChain” example listed in proposal §3.1.
Context-card store (Chatbot)	React Context (ChatContextStore) — client-side, interim	Interim implementation of the §3.3.2 “context push & storage” mechanism. Server-side contextId persistence deferred to Phase 3.
Candlestick (Chart Widget)	lightweight-charts + Financial Modeling Prep	Stock chart on the widget canvas: live OHLC from FMP, on-chart technical studies, AI commentary aligned with other OpenRouter widgets.

Component	Technology (interim)	Justification / proposal mapping
Auth, validation, hosting	Firebase, Zod, Vercel	Firebase is used instead of Clerk and Convex for authentication and cloud storage.

3.4 Evaluation Plan

Per proposal §1.5, the system is evaluated against three measurable objectives. The table below maps each objective to the module that owns it and to the metric used.

Objective	Owning module	Measurement / metric
Reduce decision latency	Widget Canvas + all widgets	Layout update latency, Chrome Lighthouse metrics, timestamp delta for automated user actions (drag start/stop, resize, save, layout switch)
Higher alert signal-to-noise	Real-Time Alerts	Alert precision and recall against a labelled event corpus; false-positive rate per active rule per trading day; end-to-end alert latency from triggering tick to client notification.
Reduce hallucination rate	Context-Aware AI Chat Panel (Chatbot)	Hallucination rate on Hallucination-bait set; grounding faithfulness on Grounded-prompt set; multi-widget coverage; streaming TTFT and inter-delta gap.

Chatbot-specific axes — the four axes listed below map directly to the proposal's acceptance criteria.

Axis	What it measures	Maps to
Hallucination rate	% of Hallucination-bait prompts where the model fabricates a number / fact instead of declining or labelling the claim as inference.	Proposal §1.5 objective "Reduce hallucination rate".
Grounding faithfulness	% of factual claims in a response that can be traced back to a pinned context card.	§3.3.4 "Grounded answers — responses cite the attached context".

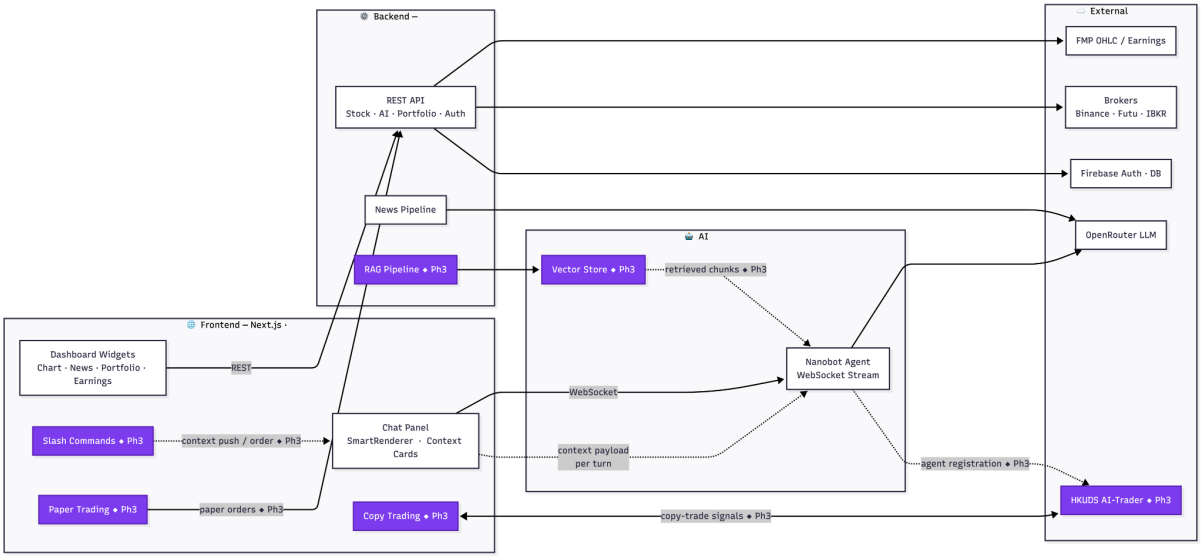
Axis	What it measures	Maps to
Multi-widget coverage	Fraction of multi-card prompts that reference all attached cards in the answer.	§3.3.4 “Multi-widget context”.
Latency / streaming	Time-to-first-token and inter-delta gap measured client-side.	§3.3.4 “Streaming responses — no perceptible gaps or failures”.

All chatbot axes are measurable with client-side instrumentation; the evaluation harness itself is a Phase 3 deliverable.

4. System Design and Implementation

4.1 System Architecture

OptiTrade Copilot follows the three-layer architecture (Frontend / Backend / AI Platform) mentioned in the proposal §3.1, with a context-management sub-system spanning all three. The frontend is a Next.js application; the backend exposes REST and WebSocket endpoints; the AI platform routes to LLM, the context-card store, and the RAG grounding engine.



(Block in Purple will be implemented in phase 3)

4.2 Module Design and Implementation

The system is composed of 3 major modules. Namely, **AI Chat Panel**, **Dynamic Widget Canvas**, and **Real-time Monitoring System (To be completed in Phase 3)**.

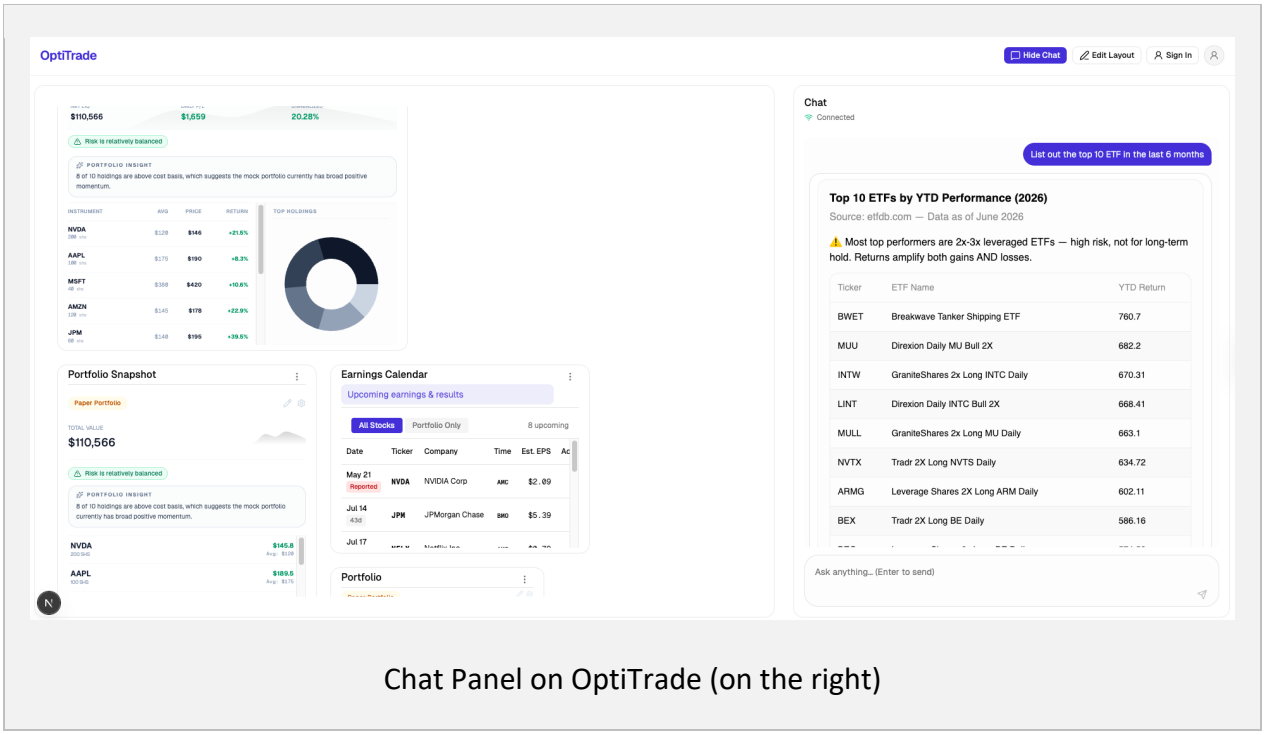
4.2.1 Context-Aware AI Chat Panel

The chat module is the primary surface through which users interact with the dashboard in natural language. It is designed around three principles: (1) streamed responses for perceived responsiveness, (2) user-controlled grounding through pinned widget snapshots, and (3) selective use of rich interactive components when a structured answer is more useful than prose.

The module is implemented on top of two open-source layers — an agentic streaming stack (Nanobot) for conversational transport and a component-rendering layer (OpenUI) for in-chat interactive output — wrapped by project-specific code that enforces the grounding and rendering rules described below.

A user can ask either an informational question, in which case the assistant replies with Markdown text, or an action-oriented or comparison question, in which case the assistant may emit a small interactive component rendered directly inside the chat. The choice

between these two modes is made by the language model based on the prompt, and the rendering layer routes the response accordingly.



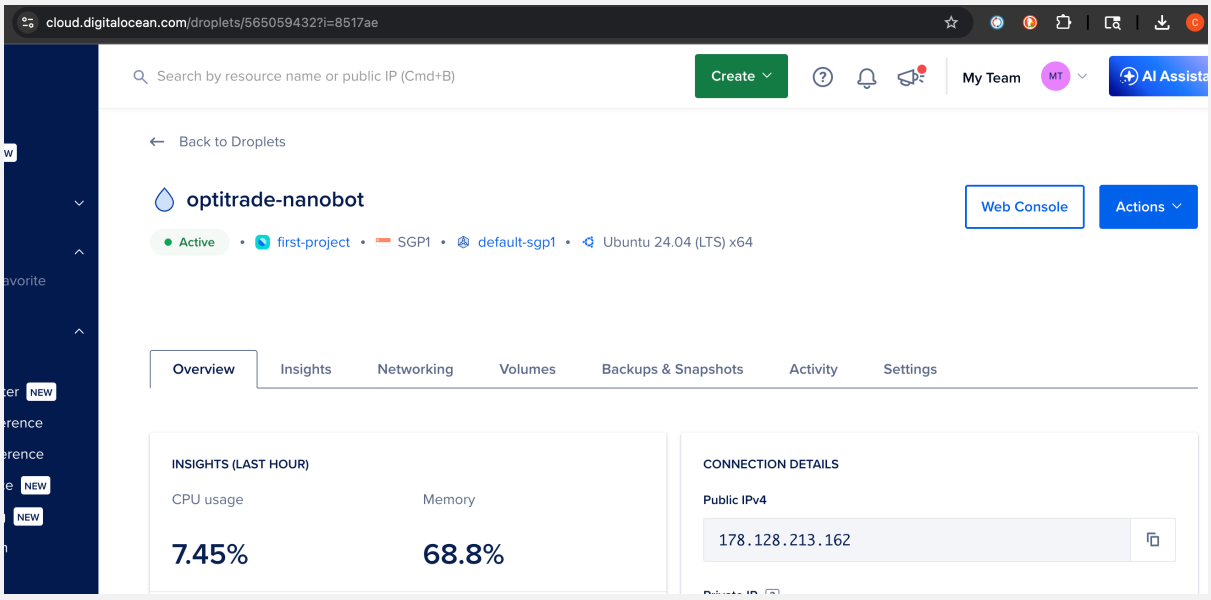
Chat Panel on OptiTrade (on the right)

4.2.1.1 Streaming Connection Layer with Nanobot

Nanobot (https://nanobot.wiki/) is an open-source agentic runtime that provides a streaming conversational transport layer for LLM-powered applications.

In OptiTrade, the Chat Panel sits directly on top of this Nanobot layer. A custom React hook encapsulates the chat connection and message state. It exposes a small interface to the rest of the user interface — the message list, connection status, a processing flag, a send action, and a manual reconnect action — and hides the streaming, reconnection, and prompt-bookkeeping logic from the rest of the application.

The hook is responsible for three behaviors. It routes incoming streamed fragments to the correct message as they arrive, so partial responses appear progressively rather than as a single block. It attaches the component-library system prompt only on the first turn of a conversation, which keeps subsequent requests lightweight and reduces token cost. It also exposes a connection state that drives both a typing indicator and a status pill, allowing the user to recover from a dropped connection without refreshing the page. This isolation is important because the conversational transport is the most failure-prone external dependency in the module, and changes to the underlying stack should not propagate into the rest of the user interface.



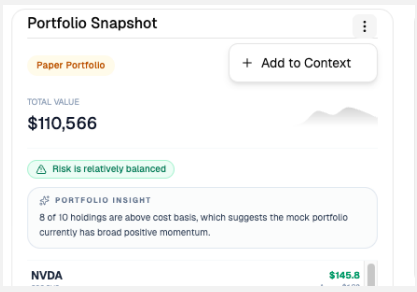
Nanobot backend hosted in digital ocean

4.2.1.2 Widget Context Registry

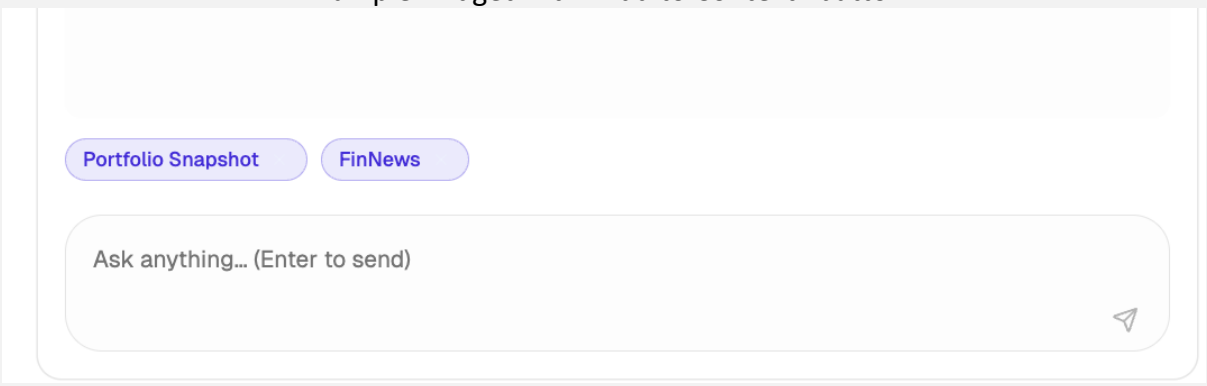
The widget context registry is a shared store that lets users pin snapshots from any dashboard widget to the chat as evidence for the next question. The registry enforces three rules that follow directly from the project's grounding goal.

The first rule is uniqueness: pinning the same widget a second time refreshes its snapshot rather than creating a duplicate entry, which prevents the same evidence from being counted twice in the prompt. The second rule is auto-clearing after send: pinned snapshots reset once a message has been dispatched, so stale evidence is not silently carried into a later, unrelated question. The third rule is plain-text serialization: each widget is responsible for producing its own textual snapshot, which keeps the registry framework-agnostic and makes the evidence package human-readable for both debugging and evaluation.

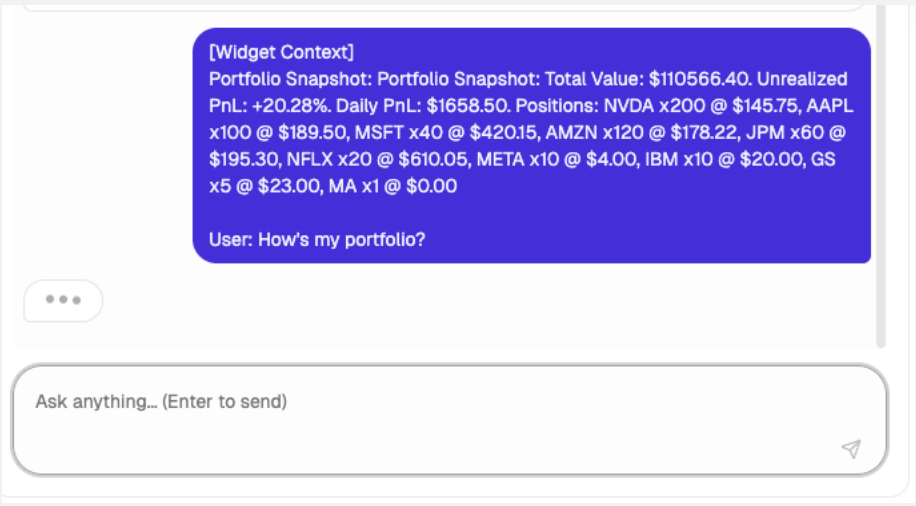
These rules together implement the project's central design choice that the user, not the assistant, decides which dashboard evidence is in scope for a given turn.



Example Widget with “Add to Context” button



Chatbot with Context Snapshot Added

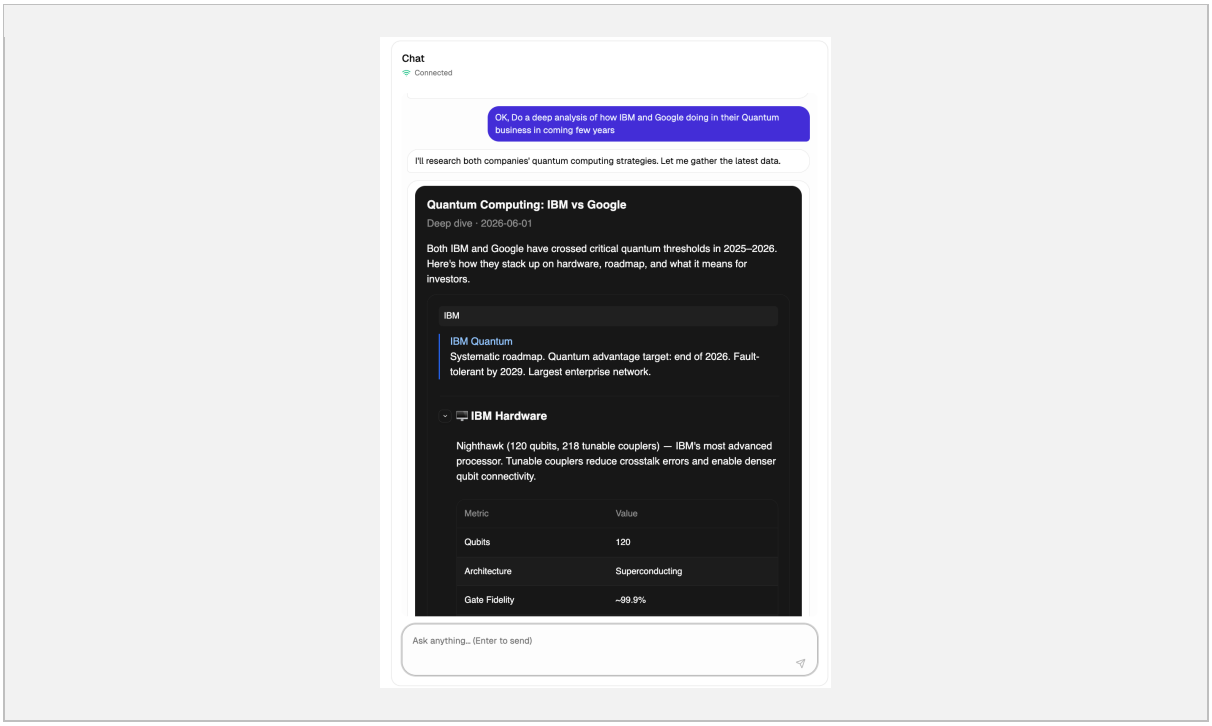


Chatbot Inferencing with Context

4.2.1.3 Chat Panel — composition root

The chat panel binds the streaming connection, the context registry, and the response renderer into a single user-facing surface. It is responsible for the message log with auto-scroll behavior, for displaying the currently pinned context cards as removable chips above the input box, for assembling the outgoing payload when at least one card is pinned, and for surfacing connection state through a status pill that doubles as a manual reconnect control during error or disconnected states. A typing indicator is shown while a response is in progress.

When evidence cards are present, the outgoing payload is prefixed with a clearly delimited widget-context block followed by the user's question, after which the registry is cleared. This delimiter convention is important for two reasons: it gives the assistant a consistent place to look for grounding material, and it allows later evaluation tooling to inspect exactly which evidence was supplied for any given turn.



4.2.1.4 SmartRenderer — Response Routing Between Markdown and Interactive Components

The component-rendering layer used here is OpenUI, an open-source library that defines a small markup language for language models to emit interactive components and a corresponding React renderer. Language-model responses therefore arrive as plain text and may contain either an ordinary **response** or a short OpenUI program. A response router inspects each completed response, separates any optional preamble text from any embedded component program, and forwards each part to the appropriate renderer. If a component program is present, the preamble is rendered as Markdown and the program is passed to the component renderer with the streaming flag forwarded so that hydration can be throttled while text is still arriving. If no component program is present, the entire response is rendered as Markdown.

This routing keeps non-UI responses visually identical to ordinary chat output while preserving the option for the model to emit richer interactive components when a structured answer is more informative — for example, a comparative table or a small action control. Because the routing decision is made on the rendered output rather than on a separate signal from the model, the design tolerates partial or malformed component programs without breaking the underlying chat flow.

4.2.1.5 Chatbot — prompt assembly

Two prompt segments are assembled on the client. The first is a *system segment* that describes the available interactive components and instructs the model to reply either in plain Markdown or as a component program; this segment is sent only on the first turn of a conversation. The second is a *per-turn context segment* that wraps the user's text inside a

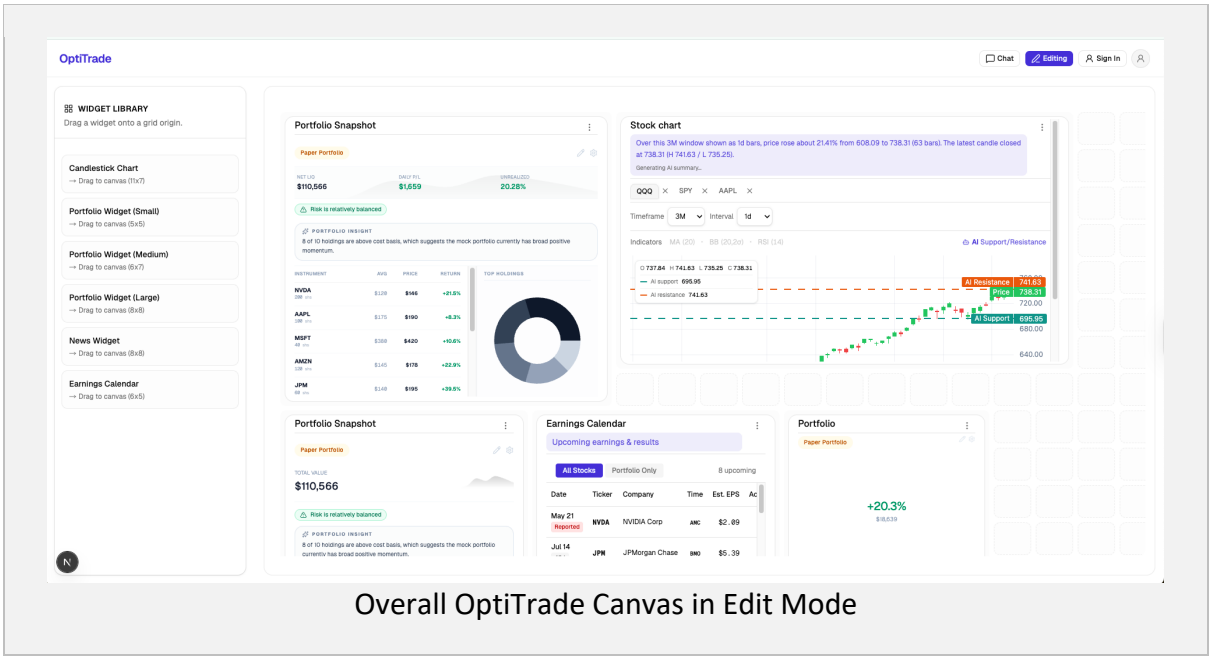
labelled widget-context block whenever at least one snapshot is pinned, so that the model is explicitly aware of which evidence it must ground against on the current turn.

A separate set of grounding instructions is layered on top of the component-library prompt to discourage unsupported claims. These instructions ask the assistant to mark any statement that is not directly supported by the pinned evidence as an inference rather than as fact, and to acknowledge missing evidence rather than fabricating it. The instructions are still being tuned and are evaluated through the prompt sets described in Section 3, particularly the missing-evidence and multi-widget cases. Refinement of these instructions is one of the remaining items in the final phase.

4.2.2 Dynamic Customizable Widget Canvas

The widget canvas is the dashboard's main visual surface and the host environment for every data widget described in the remainder of this section. It implements provides the responsive grid, drag-and-drop placement, resizing, snap behaviour, multi-layout presets, and layout persistence on which all individual widgets depend. The widgets that follow — Chart, Portfolio, and News — are tenants of this canvas: each one is a self-contained component that conforms to the canvas's layout contract and its chat-context export interface, but is otherwise independently developed and independently rendered.

- **Grid framework:** A device-aware responsive grid drives placement. It uses column-based mobile, tablet, and desktop presets with snap-to-grid alignment, breakpoint switching, and optional safe-area padding so that widgets reflow cleanly across viewports rather than being clipped or overlapped.
- **Drag, drop, and resize:** Drag and resize behaviour is wired through the grid library's native handlers, including drag handles and keyboard-driven movement for accessibility. Each widget emits a layout-change event when its position or size changes, and these events are consumed by the persistence layer.
- **Layout persistence:** A versioned JSON schema describes each layout. Drafts are kept in local storage so an in-progress arrangement survives a page reload, and committed layouts are written to Firebase as the canonical record. Versioning the schema is intentional: future canvas revisions can migrate older saved layouts without losing user work.
- **Edit-mode user experience:** A toggleable edit mode exposes a toolbar containing Add Widget, Save, Revert, Undo / Redo, and Device Preview controls. While in edit mode the canvas overlays translucent edit affordances, snap guides, and dimension handles, asks for confirmation on save, and surfaces inline help and keyboard shortcuts. Outside edit mode the canvas is read-only, which prevents accidental layout changes during normal use.



Overall OptiTrade Canvas in Edit Mode

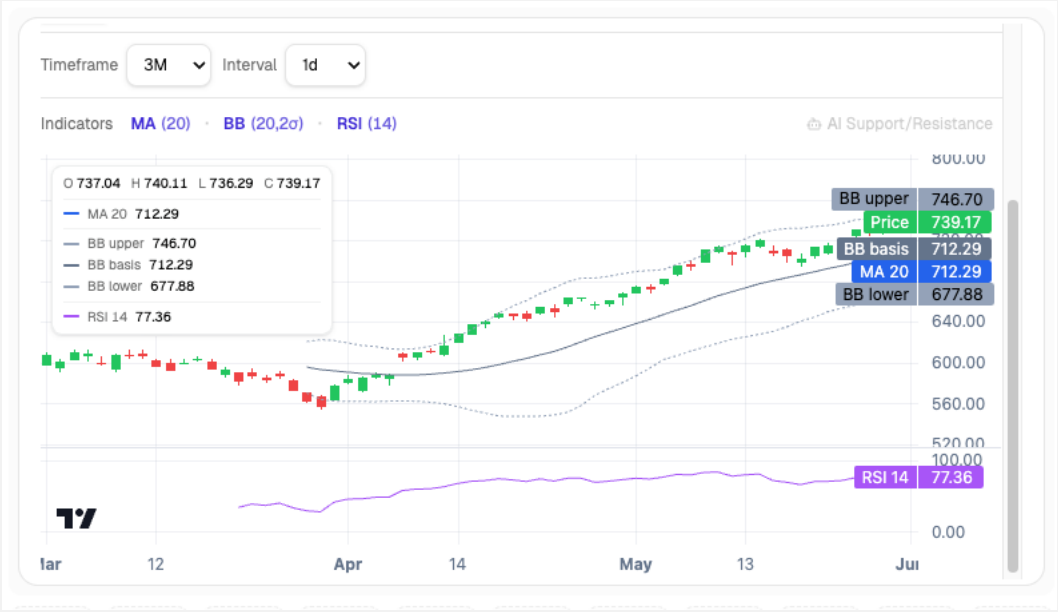
4.2.2.1 Chart Widget – Candlestick

The chart widget implements as the dashboard's interactive price-and-indicator view. It serves two roles in the system: direct visual inspection of market movement, and the export of a concise chart snapshot to the Context-Aware Chat Panel described in §4.2.1. The widget is composed of three layers — a rendering layer, a market-data bridge, and an AI analysis pipeline — described below.

Rendering and layouts: Charts are rendered using TradingView's lightweight-charts library. Candlestick series are presented inside responsive default, medium, and large layouts that conform to the canvas grid. A timeframe and interval control is presented for the user to select according to individual needs.

Market data in live mode: When live mode is active, the widget calls the FMP-backed endpoint. A frontend bridge translates the selected timeframe and interval into the backend query format, normalises the returned candles for the chart library, and caches responses keyed by symbol, timeframe, and interval so that repeated views remain responsive.

Indicators: The Phase 1 indicator set covers Moving Average, Bollinger Bands, and RSI. Overlays are toggled from the widget toolbar; RSI is shown in a separate sub-panel, and the study legend reports the latest computed values. MACD (12/26/9), volume bars, and the proposal's five-symbol watchlist target are documented as Phase 3 gaps; the interim widget supports three tickers.



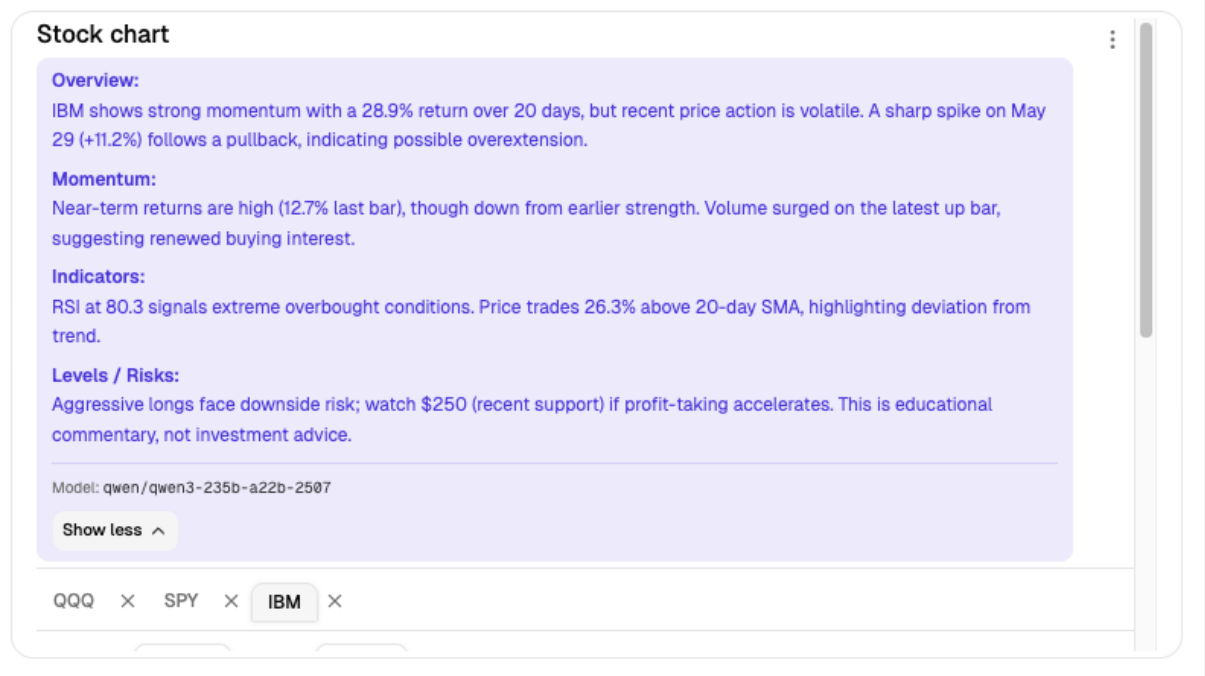
Stock Chart with various indicators activated

AI analysis pipeline: The AI analysis is designed to turn the user's current chart view into a short, explainable technical summary. It is not intended to predict prices or issue trading decisions; instead, it answers a narrower question — given the selected symbol, timeframe, interval, and recent OHLC data, what does the current momentum and indicator context suggest? This narrower framing supports the project's wider goal of reducing information overload while keeping AI output grounded in visible widget data.

The pipeline is triggered when the user changes symbol, timeframe, or interval, and results are cached per symbol / timeframe / interval combination to avoid redundant model calls. On the backend, the route does not forward raw candles to the model. It first retrieves OHLC candles from the stock chart service and derives two deterministic snapshots — a *MomentumSnapshot* (1-, 5-, and 20-bar return percentages) and a *TechnicalSnapshot* (RSI-14, SMA(20), SMA(50), and the latest close's distance from SMA(20)) — which form the factual basis of the prompt and prevent the model from inventing indicator readings.

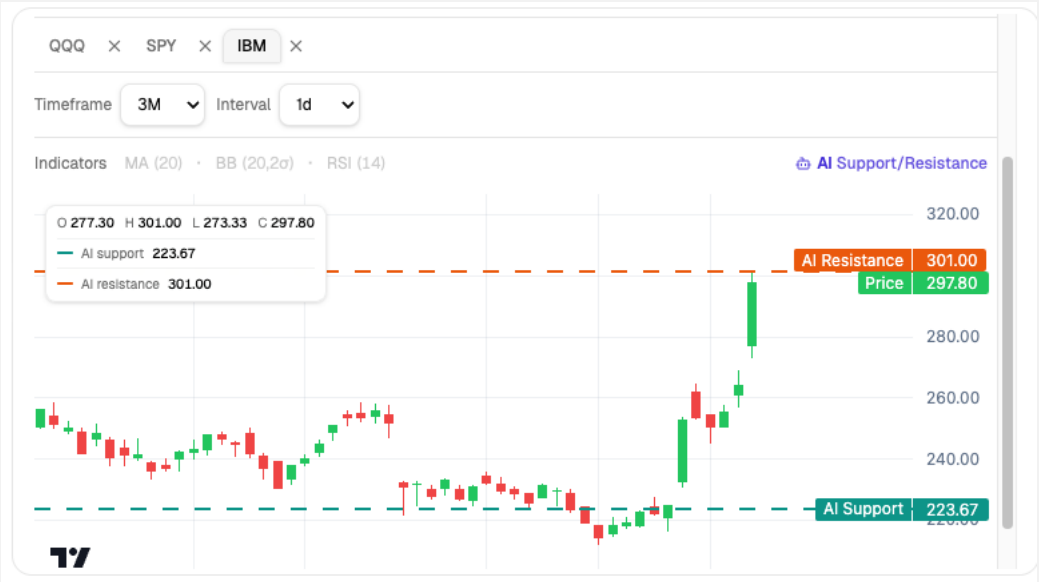
The prompt combines three inputs: the symbol and chart metadata, the metrics JSON, and a bounded recent-candle sample. The system instruction restricts the model to the provided numbers, forbids invented prices, requires a concise trend reading, and mandates explicit uncertainty when the window is short or indicators are missing. Four fixed Markdown sections — Overview, Momentum, Indicators, and Levels / Risks — keep the output predictable for the widget UI.

The proposal's explicit Overbought / Oversold / Neutral badge is not yet exposed in the UI and remains Phase 3 work.



Stock Chart AI Analysis

Support and resistance overlay: A separate support / resistance route returns deterministic pivot-cluster levels rather than LLM-generated prices. When enabled, the widget draws these levels as horizontal price lines on the chart and includes them in the chart context sent to chat.



Stock Chart AI Support and Resistance Level

Chat-context export: The chart context payload records the active symbol, watchlist, timeframe, interval, window-level momentum, enabled indicators, and support / resistance levels as plain text. This keeps the chat integration simple while giving the assistant enough evidence to answer chart-specific prompts, as demonstrated in the grounding trial.

4.2.2.2 Portfolio Widget

The portfolio widget presents a responsive Portfolio *Full view* together with compact portfolio tiles, all backed by a shared REST API contract. A FastAPI portfolio service exposes endpoints for snapshot retrieval and broker-connection settings, alongside persisted paper-portfolio and editable-portfolio routes backed by local JSON files for the interim milestone. Storing the interim state as JSON keeps the development loop fast and lets the same fixtures be replayed in Storybook and in tests, while leaving the API surface unchanged for the eventual database-backed implementation.

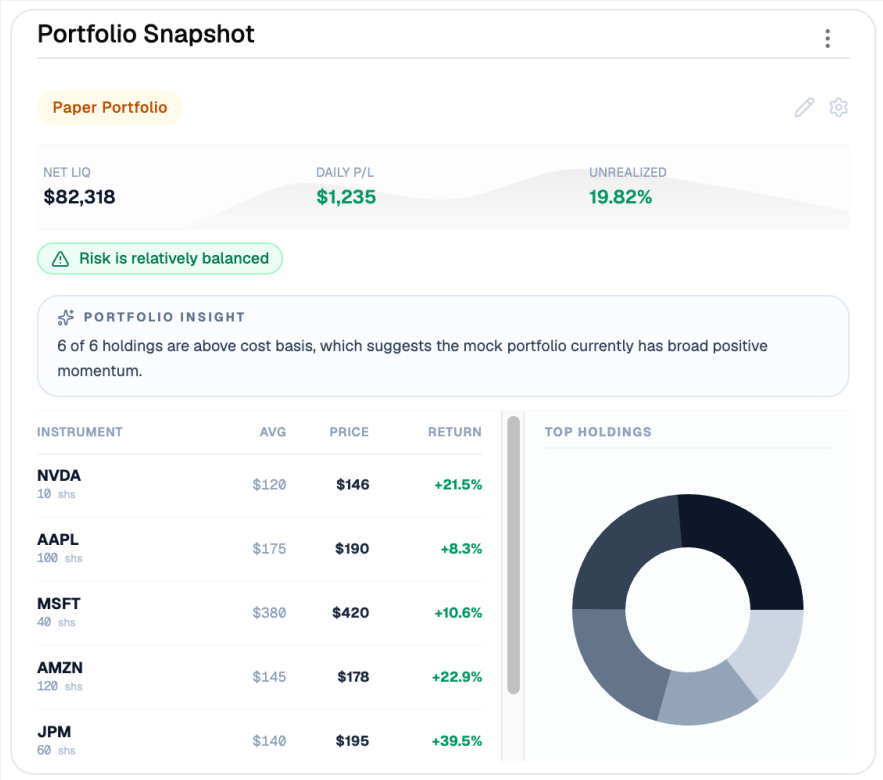
The frontend renders the portfolio across small, medium, and large layouts so that the same data source can serve a compact tile on a busy canvas and a detailed full-width view when the user wants to focus on holdings. The large layout, shown above, surfaces the metrics that drive day-to-day monitoring — net liquidation value, daily profit and loss, and unrealised return — directly under the title, followed by a per-instrument table giving average cost, current price, and per-position return. A Top Holdings donut on the right makes concentration visible at a glance, which is the visual counterpart of the concentration signal consumed by the insight layer described below. When no live broker is connected, the widget falls back cleanly to a paper / mock mode, clearly badged in the header, so that the dashboard remains usable for demonstration, screenshots, and offline development.

The first AI health-insight layer is intentionally deterministic rather than LLM-generated. It derives a single risk flag — for example Risk is relatively balanced in the screenshot — and a short insight sentence from three signals: concentration (how much of the portfolio sits in the largest positions), breadth (how many positions are above cost basis, e.g. "6 of 6 holdings are above cost basis"), and cost-basis dispersion (how far current prices have moved from average cost). Keeping this layer rule-based at the interim milestone has three practical benefits: the output is immediate, with no model latency on the critical render

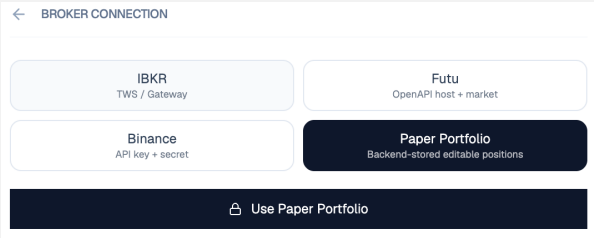
path; it is fully auditable, because every sentence can be traced to a numeric threshold; and it carries no hallucination risk, which matters for a panel that shows financial figures next to a natural-language judgement. A model-driven version is planned for Phase 3, once the deterministic baseline has been validated against user feedback and can serve as a ground-truth comparator for the LLM output.

Broker connectivity is exposed through a dedicated panel rather than buried in settings, since switching data sources is a first-class workflow during development and demonstrations. The panel offers IBKR (TWS / Gateway), Futu (OpenAPI host plus market entitlement), Binance (API key plus secret), and the backend-stored editable Paper Portfolio, with the active source highlighted and a single primary action — Use Paper Portfolio in the screenshot — confirming the selection. Only the paper-portfolio path is fully wired in the interim milestone; the live-broker tiles document the integration contract that Phase 3 will fill in, so that the UI does not have to change when real broker adapters land.

Finally, the widget participates in the dashboard's chat context in the same way as the chart and news widgets. Its chat-context export serialises the active portfolio snapshot — summary metrics, the positions table, the deterministic insight string, and broker / source metadata — into the plain-text format consumed by the chat panel, so the assistant can reason over the user's current exposure alongside other pinned widgets without re-querying the portfolio service.



Portfolio Widget Large



Broker Panel

4.2.2.3 News Widget

The news widget aggregates articles from two RSS feeds — Yahoo Finance and The Economic Times — fetching up to ten items per source on each run. Once the raw feed data arrives, the pipeline applies two filtering passes. The first pass uses a finance keyword list to discard non-financial items. The second pass compares article titles and drops near-duplicates so that the same headline does not appear twice on the dashboard.

After filtering and deduplication, each remaining article is sent to OpenRouter's cloud API with a prompt tailored for financial news. The model returns three pieces of structured information: a short executive summary, a sentiment score in the range −1 to +1, and a risk label of High, Medium, or Low. The processing runs asynchronously so that the rest of the user interface is never blocked waiting for the analysis to complete.

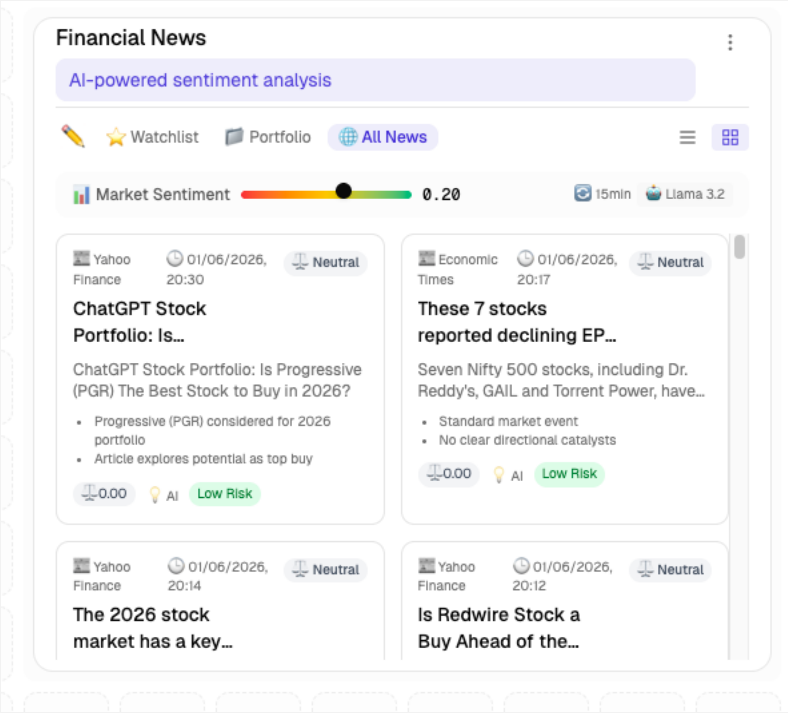
The free API tier has two practical limitations: it enforces rate limits, and it occasionally times out under heavy load. To make the widget robust against these failures, a local fallback engine based on keyword matching estimates sentiment and risk on its own when the cloud API fails or refuses a request. The fallback is less accurate than the LLM but ensures that the dashboard remains populated and that blank cards do not appear, which preserves the user's workflow even during external service degradation.

When a user clicks Add to Context on a news card, the widget serialises the article into a plain-text snapshot in the format specified by

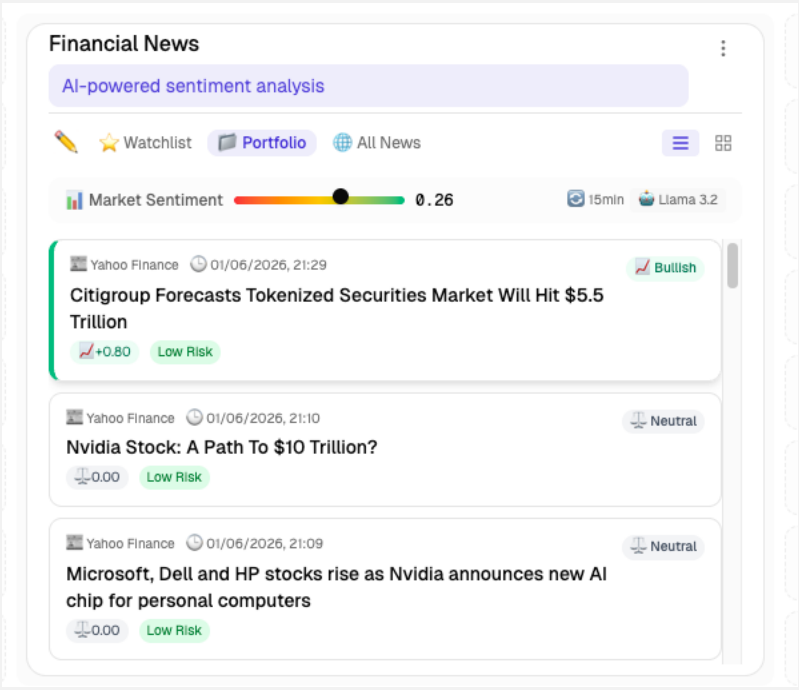
`[News Card - <Symbol>] Title: <Title> | Sentiment: <Score> | Summary: <AI Summary Text>`

— and pushes it into the chat context registry described in §4.2.1.2. The assistant can then refer to that snapshot when answering questions about the article.

For Phase 3, the plan is to add a vector store (pgvector) that indexes daily article embeddings. This will allow the assistant to run semantic searches over older news without invoking the LLM for every individual query. The structuring pipeline that produces today's article fields will feed directly into that retrieval system, so no separate ingestion path is required.



Financial News (Grid view)



Financial News (List view)

4.2.3 Real-Time Monitoring & Alerts

We will explore using nanobot backend as the market monitoring engine through cron jobs and send alerts to configured channels such as email or Slack. This will be implemented in phase 3.

4.3 Implementation Status

Status by module against proposal Phases 0–2. Phase 0 and Phase 1 line items are aggregated; Phase 2 line items are listed individually.

Module / line item	Status	% Complete	Owner
Phase 0 — proposal & infra setup	Done	100%	All
Phase 1 — Widget Canvas + drag/drop + persistence	Done	100%	Ng Wing Yin
Phase 1 — Chart Widget (price + indicators)	Done	100%	Wong Wai Wing
Phase 1 — Portfolio Full + P/L Tile + Top Mover Tile	Done	100%	Ng Sui Yat
Phase 1 — Broker mock + market data service	Done	100%	Ng Sui Yat
Phase 1 — News feed RSS aggregation	Done	100%	Hui Nga Chi
Phase 1 — Keyword-based financial news filtering	Done	100%	Hui Nga Chi
Phase 2 — AI Insights for Portfolio Full	In Progress	80%	Ng Sui Yat
Phase 2 — AI Momentum Snapshot for Chart Widget	Done	100%	Wong Wai Wing
Phase 2 — News feed aggregation + per-article AI insight	Done	100%	Hui Nga Chi
Phase 2 — Background pipeline with automated 15-minute polling loop	Done	100%	Hui Nga Chi
Phase 2 — Finance-specific prompt engineering for sentiment/risk/summary	Done	100%	Hui Nga Chi
Phase 2 — Chat Panel UI + streaming	Done	100%	Cheung Ching Nam
Phase 2 — Context management / contextId store	Done	70%	Cheung Ching Nam

Module / line item	Status	% Complete	Owner
Phase 2 — Chat Context Export across all widgets	Done	100%	Cheung Ching Nam
Phase 2 — Grounded response system + safety	In Progress	50%	Cheung Ching Nam
Phase 2 — SmartRenderer (OpenUI + Markdown routing)	Done	95%	Cheung Ching Nam
Phase 3 — Infrastructure Set up	In Progress	20%	All
Phase 3 — Server-side contextId persistence	Not Started	0%	Ng Wing Yin
Phase 3 — News-widget RAG grounding	Not Started	0%	Cheung Ching Nam + Hui Nga Chi
Phase 3 — Hallucination-rate evaluation harness	Not Started	10%	Cheung Ching Nam
Phase 3 — Alert rule authoring UI	Not Started	0%	Ng Sui Yat + Ng Wing Yin
Phase 3 — Scheduler and Email notification	Not Started	0%	Cheung Ching Nam + Hui Nga Chi

5. Schedule for the Remaining Project

This section lists the remaining work in Phase 3.

5.1 Phase 3 Timeline

Task	Owner	Start	End	Deliverable
Move grounding instructions to system prompt; add hallucination-labelling pass (chatbot)	Cheung Ching Nam	Jun 5	Jun 12	Updated system prompt + before/after eval
Server-side contextId persistence (chatbot + backend)	Cheung Ching Nam + Ng Wing Yin	Jun 8	Jun 22	Cross-session chat resumption
News-widget RAG retriever (chatbot + backend)	Cheung Ching Nam + Hui Nga Chi	Jun 15	Jul 5	News-grounded answers with citations

Task	Owner	Start	End	Deliverable
Hallucination-rate evaluation harness (chatbot)	Cheung Ching Nam	Jun 20	Jul 8	Harness + scores against benchmark question set
Alert rule authoring UI	Ng Sui Yat	Jun 10	Jun 22	Alert config UI
MACD Indicator (12/26/9) and Volume bars, and Five-symbol watchlist	Wong Wai Wing	Jun 5	Jun 10	Enhanced Candlestick Chart
Overbought / Oversold / Neutral badge for individual stock	Wong Wai Wing	Jun 10	Jun 14	Candlestick with momentum assessment
LLM-driven health-insight layer	Ng Sui Yat	Jun 5	Jun 10	Model-driven portfolio insight
Database-backed persistence	Ng Sui Yat	Jun 11	Jun 20	Persisted Portfolio in Database
End-to-end testing across all features	All	Jun 25	Jul 10	Test report
Staging and production environment setup	Ng Wing Yin	Jun 1	Jun 15	Deploy-ready environments
Performance optimisation and load testing	All	Jul 5	Jul 12	Bench results
Final documentation + project webpage update	All	Jul 8	Jul 17	Final webpage + final report
Presentation preparation (chatbot demo + slides)	All	Jul 12	Late Jul	Slides + live demo

5.2 Key Milestones

- M1 — Grounding instructions moved into system prompt; hallucination labelling implemented (Jun 12).
- M2 — Server-side contextId persistence shipped (Jun 22).
- M3 — News-widget RAG live in dev (Jul 5).
- M4 — Hallucination-rate benchmark scored (Jul 8).
- Final report submission — by Jul 17.
- Oral examination — late Jul.

6. Challenges and Risk Mitigation

6.1 System-level risks

Challenge / Risk	Likelihood	Impact	Mitigation
Market data / broker API rate limits or outages	Medium	High	Implement response caching with TTLs for frequently requested data; configure exponential backoff with jitter on retries; integrate a secondary data provider as failover; monitor rate-limit headers and alert before thresholds are hit.
Schedule slip on Phase 3 deliverables	Medium	Medium	Each module has an MVP fallback; non-critical features can be cut without breaking the integrated demo.
Integration mismatch between modules	Medium	Medium	Weekly integration syncs; shared context-card schema is documented in widget-guide.md and frozen during Phase 3.
Hosting / deployment failure on demo day	Low	High	Adding staging and production environments, versioning for quick rollback, pre-recorded demo

6.2 Chatbot-specific risks

Challenge / Risk	Likelihood	Impact	Mitigation
LLM hallucinations under multi-card prompts (P2)	Medium	High	Move grounding instructions to the system prompt; add an explicit “label as inference” instruction; measure on the Hallucination-bait set per proposal §1.5.
Nanobot WS endpoint downtime	Low	High	Existing `reconnect()` path; document a fallback to a self-hosted agent or to Azure OpenAI (per proposal §3.1) for demo day.

Challenge / Risk	Likelihood	Impact	Mitigation
OpenUI Lang parsing regressions on new model versions	Medium	Medium	Pin OpenUI library versions; keep Markdown fallback in SmartRenderer; smoke-test rejects unparsable programs gracefully.
Context-window saturation when many cards are pinned	Medium	Medium	Cap pinned cards at a configurable limit; truncate per-card text; warn the user in the chips bar.
Schedule slip on RAG news grounding	Medium	Medium	RAG is incremental — even without it, explicit context-card grounding still satisfies §3.3.4. Reduce scope to top-N retrieval if needed.
Server-side contextId store integration delay	Medium	Low	Client-side ChatContextStore already satisfies interim acceptance; server persistence can ship after final report if needed without breaking the demo.

6.3 Candlestick widget risks

Challenge/Risk	Likelihood	Impact	Mitigation
FMP market-data rate limits or outages	Medium	Medium	1. Cached responses keyed by symbol / timeframe / interval to suppress redundant calls. 2. Added exponential backoff with jitter on retries and surfaced a clear "data unavailable" state on the chart toolbar instead of a blank canvas.
AI analysis fabricating indicator readings or price targets	Medium	High	1. The route never forwards raw candles directly; it derives deterministic MomentumSnapshot and TechnicalSnapshot values that form the factual basis of the prompt. 2. The system instruction restricts the model to the supplied numbers, forbids invented prices, and requires

			explicit uncertainty when the window is short. 3. Output is constrained to four fixed Markdown sections (Overview, Momentum, Indicators, Levels / Risks) so unsupported sections cannot be silently introduced. 4. Results are cached per symbol / timeframe / interval to prevent variance across identical views.
Performance degradation with multiple chart widgets on the canvas	Medium	Medium	1. Each chart instance subscribes only to its own symbol stream; there is no global tick fan-out. 2. AI analysis is triggered only on user-driven changes (symbol, timeframe, interval) and gated by the per-key cache, so re-renders do not re-invoke the model. 3. Indicator overlays are computed incrementally on the latest candle rather than recomputed across the full window on every tick.

6.4 News widget risks

Challenge/Risk	Likelihood	Impact	Mitigation
Cloud API Quota Exhaustion & Rate Limits	High	Medium	1. Implemented a strict sliding-window backend cache and deduplication layer to skip identical updates. 2. Extended the cron worker's automated polling cycle to 15 minutes to pace requests.
News Source Policy Shifts & Anti-Scraping Blocks	Medium	High	1. Abandoned brittle full-text HTML scraping in favor of standardized, highly reliable RSS syndication XML feeds. 2. Utilized standard HTTP headers (e.g., rotating User-Agent strings) and encapsulated the fetching loop in a try-except

			block with detailed logging to prevent daemon thread crashes. 3. Structured the system to allow seamless multi-source redundancy; if one feed fails, fallback feeds ensure the dashboard remains populated.
Cross-field Logical Inconsistency in LLM Outputs	Medium	Medium	1. Engineered explicit, strict hierarchical constraint instructions within the system prompt. 2. Built a post-processing "Physical Guardrail" logic layer in the backend Python code to automatically cross-validate and force alignment (e.g., if sentiment is within ±0.01, risk is strictly overridden to "Low Risk"; if risk is "High Risk" and sentiment is non-negative, sentiment is mathematically forced to -0.50).
Phase 3 Vector Storage & Retrieval Scaling Bottleneck	Low	High	1. We will deploy pgvector within our PostgreSQL instance and build HNSW (Hierarchical Navigable Small World) or IVFFlat indexes optimized for high-dimensional financial text embeddings. 2. Implement a strict time-based partitioning strategy, allowing the Chatbot to prune search spaces by prioritizing news embeddings from the last 7 to 30 days during real-time retrieval.

6.5 Portfolio widget risks

Challenge/Risk	Likelihood	Impact	Mitigation
Broker Mock vs Real Broker Schema Drift	Medium	High	1. Standardized one portfolio snapshot contract across frontend and backend. 2. Added source labelling for paper/mock vs live. 3. Covered the API with contract tests for future broker adapters.

Incorrect or Stale P/L After Portfolio Edits	Medium	High	1. Persisted editable portfolios in the backend. 2. Recomputed value and P/L from saved positions on reload. 3. Persisted mock history so the chart stays consistent after refresh.
Broker Connectivity Instability in Interim Integration	Medium	Medium	1. Added broker-connection UI with fallback to paper mode. 2. Isolated connection handling behind /api/portfolio/connect. 3. Kept the backend-backed mock path as the default interim source.
Portfolio Insight Layer Too Shallow for Expert Users	Medium	Medium	1. Used deterministic insight rules for concentration and breadth. 2. Kept outputs framed as guidance, not advice. 3. Deferred richer LLM reasoning to Phase 3.

7. Task Completed and Learning Hours

Member	Role	Tasks Completed	Hours
Wong Wai Wing (3036383513)	Stock Data lead – Candlestick Widget, Stock Data API	Interactive candlestick widget with three-ticker watchlist, timeframe and interval controls, and responsive sizing across small/medium/large layouts; Moving Average, Bollinger Bands, and RSI overlays with corresponding backend technical snapshots; OHLC ingestion and caching layer behind the stock-data REST endpoints;	~190
Ng Wing Yin (3036383927)	Widget canvas lead - Dynamic Customizable Widget System, Infrastructure and DevOps	Nx monorepo and Git version control setup; Dynamic widget canvas mechanism and UI/UX design; Storyboard widget prototypes and stories; Establish connectivity between frontend and backend using REST and gRPC.	~200
Hui Nga Chi (3036198774)	News Widget lead- Multi-source Financial News Aggregator	Multi-source RSS aggregation ; keyword-based financial filtering and title-based deduplication; OpenRouter cloud LLM (llama3.2:3b) integration with finance-specific prompt; local keyword-based fallback engine for API rate-limit resilience; cross-field consistency guardrail; background daemon with 15-minute polling loop;	~190
Ng Sui Yat (3036383549)	Portfolio lead — Portfolio Widget and broker-mock API path	Portfolio Full widget across small/medium/large layouts; holdings and P/L calculations; broker-connection panel; backend portfolio API integration; persistent paper-portfolio editing and mock-	~200

Member	Role	Tasks Completed	Hours
		history save/reload; top-holdings visualisation; first-pass portfolio insight card and risk flag; interim report portfolio write-up.	
Cheung Ching Nam (3036382959)	Chatbot lead — Context-Aware AI Chat Panel	Nanobot setup and hosting; useNanobot streaming WebSocket client; ChatContextStore + Add-to-Context wiring across all widgets; multi-context cards bar; SmartRenderer (OpenUI Lang + Markdown routing); per-turn [Widget Context] prompt assembly.	~190

8. Conclusion

At the interim milestone, OptiTrade Copilot has completed proposal Phases 0–1 and is towards the end of Phase 2. Across the system, the architecture and module boundaries set out in the proposal have held up under integration; no fundamental redesign is required for Phase 3.

Module-by-module summary:

- Widget Canvas — The dynamic customizable widget canvas is fully working and running. For phase 3, automated user controls will be performed to reveal potential bugs and add support for multiple widget layout preferences for the same user.
- Chart Widget — The interactive price chart is working end-to-end, with multi-timeframe selection, symbol switching, and standard overlays (moving averages, volume) rendering against live market data through the shared data layer. Phase 3 will add richer technical-analysis tooling and expose chart state to the AI Chat Panel for context-aware answers.
- Portfolio Widget — the core widget, broker-mock API path, persistent paper portfolio, and first-pass portfolio insight layer are working end-to-end; Phase 3 will focus on real broker integration and richer LLM-grounded portfolio reasoning.
- News Widget — The news pipeline is fully working and running in the background. It pulls from multiple sources, runs the articles through cloud LLM for sentiment analysis, and falls back to local keyword rules whenever the API hits rate limits or times out.
  This hybrid approach turned out to be a good call. The fallback keeps the dashboard from showing empty states, and it means we don't have to worry about API quotas becoming a single point of failure.
  We're also setting ourselves up for Phase 3. The same pipeline structures the article data in a way that will feed into the RAG system later, so the chat module will be able to pull relevant news context when needed.
- Context-Aware AI Chat Panel (Chatbot) — working end-to-end prototype satisfying four of the five §3.3.4 MVP acceptance criteria. The fifth (grounded answers with explicit inference labelling) is implemented as best-effort prompt instruction and is tracked for system-prompt-level upgrade in Phase 3. The user-pinned context cards are sufficient to operationalize the §3.3.2 grounding mechanism without requiring a full retrieval pipeline up front — has been validated on the four §3.3.4 trial patterns.
- Real-Time Alerts — It will be a focus of Phase 3 in delivering the alerts to the end user.

Remaining risks are time-boxed against Phase 3 schedule. Confidence in delivering the full proposal scope on or before the final report deadline is high.

References

Numbered citations are kept consistent with the original Detailed Project Proposal so that markers in this report ([5][6][7][8][9][10] in particular) refer to the same sources.

[1] Ramdani et al. (2025). Dynamic Dashboard Optimization with AI: Enhancing User Experience and Improving Decision Making Process. IEEE ICOA 2025. DOI: 10.1109/ICOA66896.2025.11236939.

[2] Bernales, Valenzuela & Zer (2023). Effects of Information Overload on Financial Markets: How Much Is Too Much? Federal Reserve IFDP No. 1372. DOI: 10.17016/ifdp.2023.1372.

[3] Zafar et al. (2025). From Digital Tools to Decision Outcomes: Understanding Investment Quality in FinTech Environments. DOI: 10.63056/acad.004.04.1457.

[4] Myllynen & Kamau (2025). The Evolution of Dashboard Design: Best Practices for Data Visualization in Decision Support Systems. Computational Engineering Journal. DOI: 10.54660/.ijeca.2025.1.6.01-17.

[5] Doshi-Velez & Kim (2017). Towards a Rigorous Science of Interpretable Machine Learning. arXiv:1702.08608.

[6] Kang & Liu (2023). Deficiency of Large Language Models in Finance: An Empirical Examination of Hallucination. arXiv:2311.15548.

[7] Hiriyanna & Zhao (2025). Multi-Layered Framework for LLM Hallucination Mitigation in High-Stakes Applications. Computers (MDPI). DOI: 10.3390/computers14080332.

[8] Wang & Li (2025). Artificial Intelligence for Quantitative Finance: A RAG-Augmented Multi-Agent Framework. ACM. DOI: 10.1145/3788149.3788249.

[9] Zeng (2025). QuantMCP: Grounding Large Language Models in Verifiable Financial Reality. arXiv:2506.06622.

[10] Sinha, Agarwal & Malo (2025). FinBloom: Knowledge Grounding Large Language Model with Real-time Financial Data. Knowledge-Based Systems. DOI: 10.1016/j.knosys.2026.115559.

[11] Li & Huang (2026). Real-time Multithreaded Stock Monitoring and Alert System with Enterprise WeChat Notification and Threshold Prediction. SPIE. DOI: 10.1117/12.3104110.

[12] Jiang & Zeng (2025). Financial Sentiment Analysis Using FinBERT with Application in Predicting Stock Movement. arXiv:2306.02136.

[13] Kirtac & Germano (2024). Sentiment Trading with Large Language Models. Finance Research Letters. DOI: 10.1016/j.frl.2024.105227.

Appendix A — Project Webpage

https://onenylxus.github.io/optitrade